

Python Topic 5 — Lists

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert



Layer 1 — New to Python

A **list** is an ordered collection of items — Python's version of a JavaScript array.

```
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", True, 3.14]      # can hold different types!
```

Grab items by **position** (starting at 0):

```
fruits[0]      # "apple"   → first item
fruits[1]      # "banana"  → second item
fruits[-1]     # "cherry"  → last item (negative from the end)
```

Add and remove items:

```
fruits.append("mango")      # add to the end
fruits.remove("banana")     # remove by value
len(fruits)                 # how many items
```



Layer 2 — Intermediate (real code + how it works)

Creating and accessing

```
nums = [10, 20, 30, 40, 50]
nums[0]      # 10
nums[-1]     # 50
nums[1:3]    # [20, 30] → slicing works just like strings! [start:stop]
nums[::-1]   # [50, 40, 30, 20, 10] → reversed
```

Methods you'll use constantly

```
nums.append(60)      # add ONE item to end          → JS push()
nums.pop()           # remove & return LAST item    → JS pop()
nums.pop(0)          # remove & return item at index 0
```

```

nums.insert(1, 99)      # insert 99 at index 1
nums.remove(30)         # remove first matching VALUE (not index!)
nums.extend([70, 80])   # add multiple items           → JS concat/spread
nums.sort()             # sort in place
nums.reverse()          # reverse in place
nums.index(40)          # find the index of a value
nums.count(20)          # how many times 20 appears

```

JS → Python method map

JavaScript	Python
<code>arr.push(x)</code>	<code>arr.append(x)</code>
<code>arr.pop()</code>	<code>arr.pop()</code>
<code>arr.shift()</code>	<code>arr.pop(0)</code>
<code>arr.unshift(x)</code>	<code>arr.insert(0, x)</code>
<code>arr.length</code>	<code>len(arr)</code>
<code>arr.includes(x)</code>	<code>x in arr</code>
<code>arr.indexOf(x)</code>	<code>arr.index(x)</code>
<code>[...a, ...b]</code>	<code>a + b</code>

Membership check

```

if "apple" in fruits:      # → JS: fruits.includes("apple")
    print("found it")

```

Looping (no C-style index loop)

```

for fruit in fruits:       # iterate values directly
    print(fruit)

for i, fruit in enumerate(fruits): # need the index too? use enumerate
    print(i, fruit)         # 0 apple, 1 banana, ...

```

✅ **Pythonic:** `enumerate` gives both index and value — cleaner than JS `forEach((val, i) => ...)`.



Layer 3 — Expert (internals & gotchas)

1. The copy-vs-reference trap (connects to Topic 1)

```
a = [1, 2, 3]
b = a           # b is NOT a copy – points to the SAME list
b.append(4)
print(a)        # [1, 2, 3, 4] → a changed too!
```

To make a real copy:

```
b = a.copy()    # shallow copy → JS [...a]
b = a[:]        # also a copy (full slice)
b = list(a)     # also a copy
import copy
b = copy.deepcopy(a) # deep copy (for nested lists)
```

2. Shallow vs deep copy — the nested trap

```
a = [[1, 2], [3, 4]]
b = a.copy()    # shallow – outer new, inner lists SHARED
b[0].append(99)
print(a)        # [[1, 2, 99], [3, 4]] → inner changed in both!
```

⚠ **Note:** a shallow copy duplicates the outer list but inner objects are still shared. For fully independent nested data, use `copy.deepcopy()`. Matters when cloning config objects or nested API responses.

3. List comprehensions — replaces map/filter

```
# JS: nums.map(n => n * 2)
doubled = [n * 2 for n in nums]

# JS: nums.filter(n => n > 20)
big = [n for n in nums if n > 20]

# combined: map + filter together
result = [n * 2 for n in nums if n > 20]

# nested
pairs = [(x, y) for x in [1, 2] for y in [3, 4]] # [(1,3), (1,4), (2,3), (2,4)]
```

✓ **Idiomatic:** comprehensions are faster than loops and considered idiomatic Python. You'll use these everywhere.

4. The mutable default argument trap (recall Topic 1)

```
def add_item(item, lst=[]):    # 🚨 the [] is shared across all calls!
    lst.append(item)
    return lst

add_item(1)    # [1]
add_item(2)    # [1, 2] ← not [2]!
```

🔧 Fix with `None` :

```
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

5. `sort()` vs `sorted()` — mutates vs returns new

```
nums.sort()          # sorts IN PLACE, returns None
new = sorted(nums)    # returns a NEW sorted list, original untouched

# sort with a key (like JS .sort((a,b) => ...))
words.sort(key=len)    # sort by length
words.sort(key=str.lower) # case-insensitive
nums.sort(reverse=True) # descending
```

⚠️ **Common bug:** `x = nums.sort()` sets `x` to `None`, because `sort()` returns nothing. Use `sorted()` if you want the result.

6. Unpacking — a clean Python trick

```
a, b, c = [1, 2, 3]    # a=1, b=2, c=3
first, *rest = [1, 2, 3, 4] # first=1, rest=[2, 3, 4] (star grabs the rest)
```

🎯 Interview Q&A Cards

Q: `b = a` vs `b = a.copy()` for a list?

A: `b = a` binds another name to the same list — mutations show in both. `b = a.copy()` (or `a[:]`, `list(a)`) makes a new shallow copy. Nested data needs `copy.deepcopy()`.

Q: Shallow vs deep copy?

A: Shallow duplicates the outer container but shares inner objects. Deep (`copy.deepcopy()`) recursively clones everything, so nested structures are fully independent.

Q: `sort()` vs `sorted()` ?

A: `sort()` sorts in place and returns `None` . `sorted()` returns a new sorted list and leaves the original unchanged. Both accept `key=` and `reverse=` .

Q: What's a list comprehension and why use it?

A: A concise `[expr for item in iterable if cond]` syntax that replaces map/filter. It's faster than an equivalent loop and idiomatic Python.

🔗 **Memory hook:** `b = a` shares, `b = a[:]` copies. `sort()` changes in place (returns `None`), `sorted()` hands you a new one.

✅ **One-liner for interviews:** "Lists are mutable, ordered sequences. Assignment shares a reference — use `.copy()` / `[:]` for shallow, `copy.deepcopy()` for nested. List comprehensions replace map/filter and are faster and idiomatic. `sort()` mutates and returns `None`; `sorted()` returns a new list."